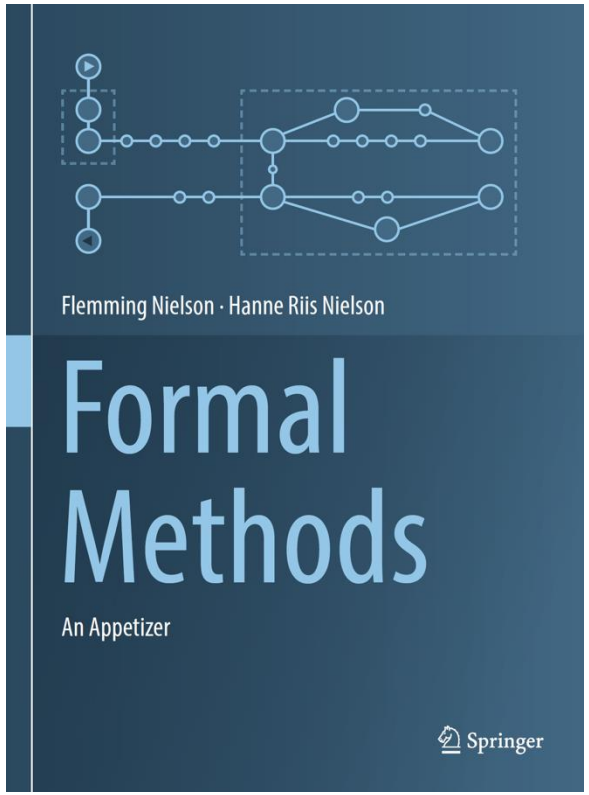
02141 – Computer Science Modelling

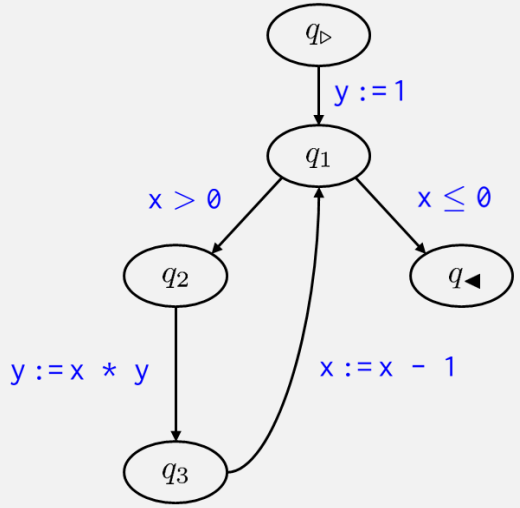
Formal Methods 02

Alberto Lluch Lafuente

(based on slides by Christoph Matheja)



Previously...



program graph modelling flow of control

σ: **Var** $\to$ **Int**

$$\mathcal{S}[\![\texttt{y := 1}]\!](\sigma) = \sigma[\mathsf{y} \mapsto 1]$$

$$\mathcal{S}[\![\texttt{x > 0}]\!](\sigma) = \begin{cases} \sigma, & \text{if } \sigma(\mathsf{x}) > 0 \\ \text{undefined}, & \text{if } \sigma(\mathsf{x}) \leq 0 \end{cases}$$

$$\mathcal{S}[\![\texttt{x} \leq \texttt{0}]\!](\sigma) = \begin{cases} \sigma, & \text{if } \sigma(\mathsf{x}) \leq 0 \\ \text{undefined}, & \text{if } \sigma(\mathsf{x}) > 0 \end{cases}$$

$$\mathcal{S}[\![\texttt{x := x - 1}]\!](\sigma) = \sigma[\mathsf{x} \mapsto \sigma(\mathsf{x}) - 1]$$

$$\mathcal{S}[\![\texttt{y := x * y}]\!](\sigma) = \sigma[\mathsf{y} \mapsto \sigma(\mathsf{x}) * \sigma(\mathsf{y})]$$

semantic function modelling how data stored in memory is updated

$$\langle q_{\triangleright}; (3,7)\rangle \xRightarrow{\texttt{y := 1}} \langle q_1; (3,1)\rangle$$
$$\xRightarrow{\texttt{x>0}} \langle q_2; (3,1)\rangle \xRightarrow{\texttt{y := x*y}} \langle q_3; (3,3)\rangle$$
$$\xRightarrow{\texttt{x := x-1}} \langle q_1; (2,3)\rangle \xRightarrow{\texttt{x>0}} \langle q_2; (2,3)\rangle$$
$$\xRightarrow{\texttt{y := x*y}} \langle q_3; (2,6)\rangle \xRightarrow{\texttt{x := x-1}} \langle q_1; (1,6)\rangle$$
$$\xRightarrow{\texttt{x>0}} \langle q_2; (1,6)\rangle \xRightarrow{\texttt{y := x*y}} \langle q_3; (1,6)\rangle$$
$$\xRightarrow{\texttt{x := x-1}} \langle q_1; (0,6)\rangle \xRightarrow{\texttt{x>0}} \langle q_{\blacktriangleleft}; (0,6)\rangle$$

operational semantics modelling all program behaviors step by step

- A deterministic system computes the same values by performing the same actions *whenever it is started on the same input*.
- An evolving system can always perform execution steps unless it reached a final configuration.

$$\langle q_{\triangleright}; (3,7)\rangle \Longrightarrow^* \langle q_{\blacktriangleleft}; (0,6)\rangle$$

"reachable via execution steps"

Making it formal

Definition. A **program graph** is a tuple

$\mathbf{PG} = (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$, where

- $\mathbf{Q}$ is a finite set of nodes,
- $q_{\triangleright} \in \mathbf{Q}$ is the initial node,
- $q_{\blacktriangleleft} \in \mathbf{Q}$ is the final node,
- $\mathbf{Act}$ is a set of actions,
- $\mathbf{E} \subseteq \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$ is a finite set of edges.

Mem is the set of all **memories**

$$\sigma : \mathbf{Var} \to \mathbf{Int}$$

Definition (**semantic function**).

$$\mathcal{S}[\![\cdot]\!] : \mathbf{Act} \to (\mathbf{Mem} \hookrightarrow \mathbf{Mem})$$

Definition. We can perform an **execution step**

$$\langle q_{\circ}; \sigma \rangle \overset{\alpha}{\Longrightarrow} \langle q_{\bullet}; \sigma' \rangle$$

whenever

- $(q_{\circ}, \alpha, q_{\bullet}) \in \mathbf{E}$ and
- $\mathcal{S}[\![\alpha]\!](\sigma) = \sigma'$ (in particular, $\mathcal{S}[\![\alpha]\!](\sigma)$ is defined).

Definition. The **operational semantics** of a program graph and its semantic function is the *set of all complete execution sequences*

$$\langle q_{\triangleright}; \sigma \rangle \overset{\omega}{\Longrightarrow}^{*} \langle q_{\blacktriangleleft}; \sigma' \rangle .$$

Recap exercise

- Construct program graphs and the semantics function for the following programs

```
x := x + 1;
x := x + 1
```

```
while (x < y) {
  x := x + 1
}
```

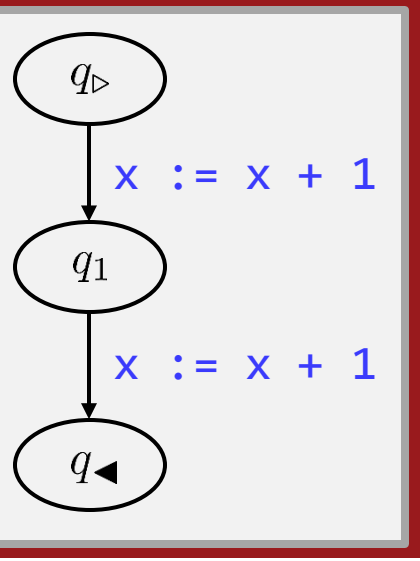
```
if (x < y) {
  x := x + 1
} else {
  skip
}
```

Solution

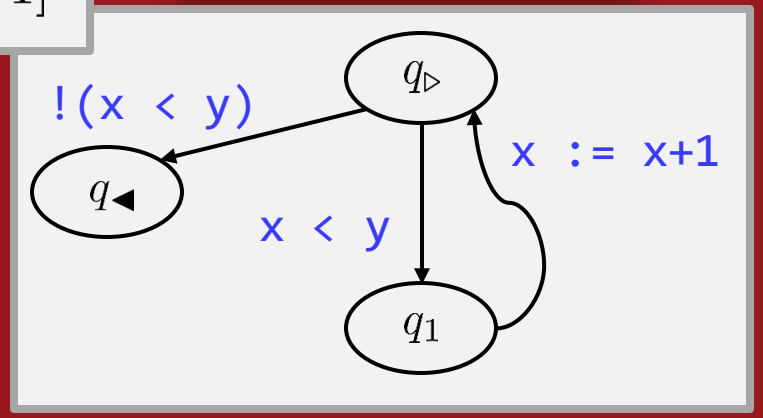
- Observation: constructing program graphs is quite mechanical

```
x := x + 1;
x := x + 1
```

$$\mathcal{S}[\![\, \mathsf{x := x + 1} \,]\!](\sigma) \;=\; \sigma[\mathsf{x} \mapsto \sigma(\mathsf{x}) + 1]$$

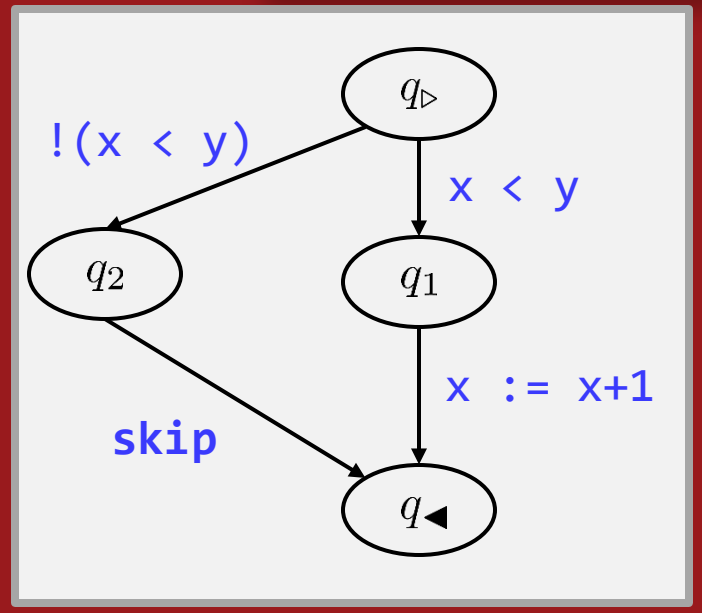



```
while (x < y) {
  x := x + 1
}
```



$$\mathcal{S}[\![\, \mathsf{x < y} \,]\!](\sigma) \;=\; \begin{cases} \sigma, & \text{if } \sigma(\mathsf{x}) < \sigma(y) \\ \text{undefined}, & \text{if } \sigma(\mathsf{x}) \geq \sigma(y) \end{cases}$$

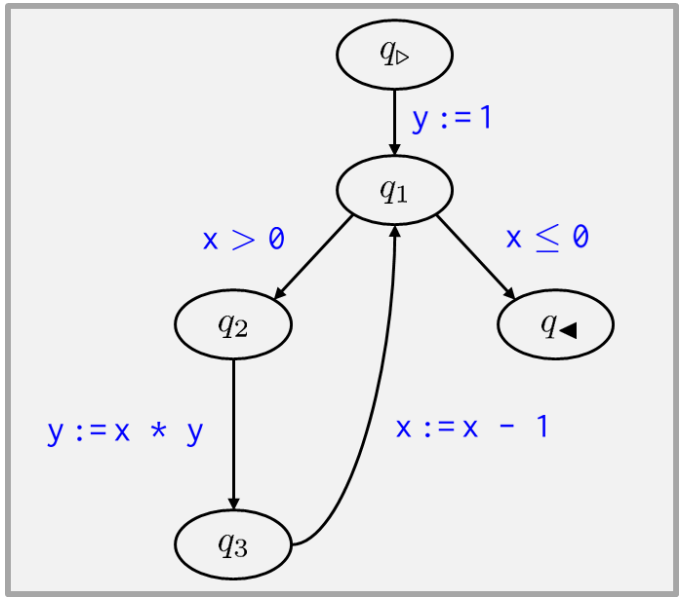
```
if (x < y) {
  x := x + 1
} else {
  skip
}
```



Programming Languages

- We develop software using programming languages
  - Examples: C, Java, F#, Haskell, Rust, ...
  - Not: program graphs
- **But:** the precise meaning of similar programming constructs in different languages can be subtle
  - Arrays start at...
    - index 0: C, Java, F#, Rust
    - index 1: Fortran, Julia, Lua, R

```
y := 1;
while (x > 0) {
  y := x * y; x := x - 1
}
```




```
int factorial(int n) {
  int result = 1;
  for (int i = 1; i <= n; ++i)
    result *= i;
    return result;
}
```

returns n!

```
def factorial(n):
  result = 1
  for i in range(1, n+1):
    result *= i
    return result
```

returns 1

Program Semantics

- **Goal:** formalize the meaning of programs using operational semantics
  - Foundation for reasoning about programs

- Approach is applicable to many programming languages
  - Lectures & Asssignment:

    Guarded Command Language

- Real-world application: verified compilers

CompCert proves that the operational semantics of a C program is equal to the operational semantics of the assembly code produced by the compiler

Formalization Approach

program

⇩

compiler

⇩

program graph

⇩

interpreter

⇩

execution sequences

FormalMethods.dk/fm4fun

fm4fun | About | Download | Environments | Formal Methods

Input Program

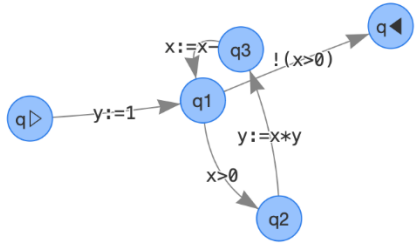
Examples

Extended Guarded Commands Code

```
y:=1;
do x>0 -> y:=x*y;
          x:=x-1
od
```

Program Graph

Non-det. | Det.



Step-wise Execution

Explore how the memory changes, upon execution of the actions. Recall that if more guards are satisfied then it is non-deterministic which action to take.

When using the **Show Trace** feature, non-deterministic choices will be shown in the first column as ⤮, and if clicked the trace will be "reset" from that configuration.

When using the **Show Resulting Configurations** feature, all non-deterministic choices will be explored, until it reaches the number of steps, gets stuck or terminates. A short summary of the final configurations is found at the end. **Warning** If the input code is highly non-deterministic, and the number of steps is high, the tree of possible end-configurations will grow big (and therefore cause long execution).

For both outputs, all non-terminated and non-stuck configurations can be extended with 1-1000 steps.

Initialization of Variables and Arrays

x = 3, y = 0

Number of Steps

10

Show Trace | Show Resulting Configurations

	Configuration				
		Memory			
Steps	Node	x	y	Status	
	q▷	3	0	Initial	
$\Rightarrow^{10}$	q1	0	6	Show More	10 +

1 trace can **Show More** after 10 steps.

Agenda

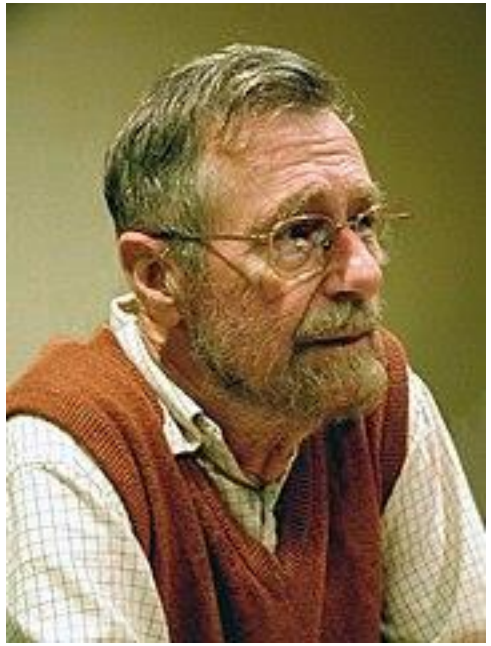
1. The Guarded Command Language
2. From Guarded Commands to Program Graphs
3. Semantics of Guarded Commands
4. Extensions (Friday)

02141 – Computer Science Modelling

THE GUARDED COMMAND LANGUAGE ➔ SYNTAX

Why Guarded Command Language?

- Initially proposed by Dijkstra
- Not used in production code
- But: good for studying principles
  - Covers many interesting aspects of reasoning about programs
  - Small enough such that we can be formal
  - Easy to extend



Edsger W. Dijkstra
(Turing Award 1972)

Guarded Commands – Examples

```
// factorial function
y := 1;
do
    x > 0 -> y := x * y;
             x := x - 1
od
```

```
// maximum function
if x >= y -> z := x
[] y > x  -> z := y
fi
```

Guarded commands form an inductive set.

Atomic Commands C.

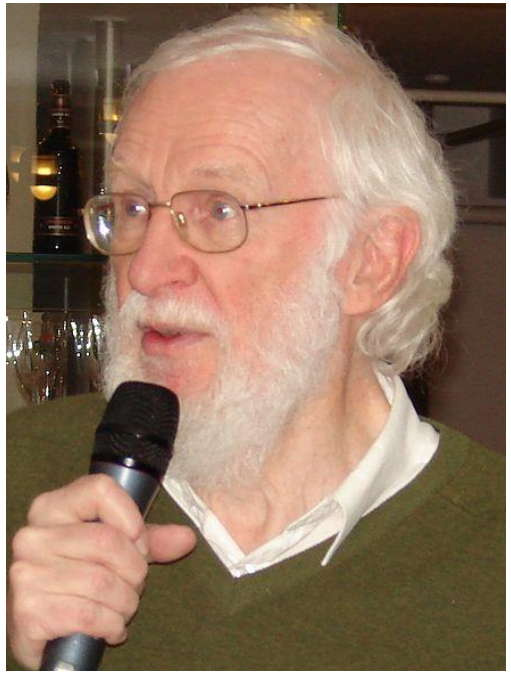
- assignments: `x := a` (a is an *arithmetic expression*)
- no operation: `skip`

Composite Commands C.

- sequencing: `C1; ...; Cn`
- conditionals: `if b1 -> C1 [] ... [] bn -> Cn fi`
  - `b1, ..., bn` are *Boolean expressions* called *guards*
  - GC1 `[]` GC2 is called the *choice* operator
- iteration: `do b1 -> C1 [] ... [] bn -> Cn od`

Guarded Commands – Syntax

- The syntax of a programming language is an inductive set
- Defined recursively via a Backus-Naur Form grammar



Peter Naur, 1928 - 2016
only Danish Turing Award winner (2005)
(the reason why “computer science”
is called *datalogi* in Danish)

Guarded Commands – Syntax

```
if x >= y -> z := x
[] y > x  -> z := y
fi
```

- The syntax of a programming language is an inductive set
- Defined recursively via a Backus-Naur Form grammar

Syntax of Commands C and Guarded Commands GC.

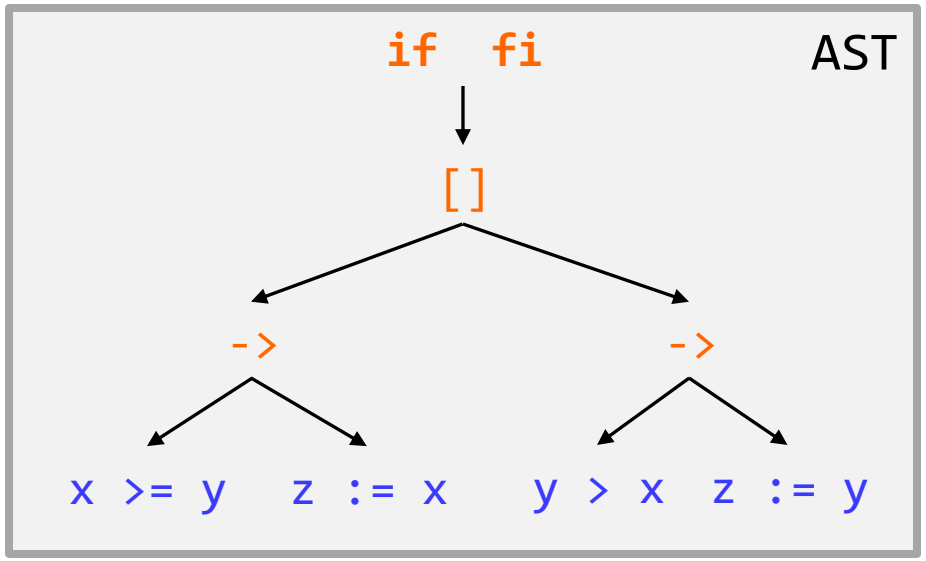
```
C  ::= x := a                    (assignments are atoms)
     | skip                      (no operation is an atom)
     | C1; C2                    (sequential composition)
     | if GC fi                  (conditional)
     | do GC od                  (iteration)

GC ::= b -> C                    (guarded commands)
     | GC1 [] GC2                (choice)
```

C can be replaced by any right-hand side of ::= separated by |

BNF grammars specify **abstract syntax trees** (ASTs)

- Nodes: syntactic operators
- Leaves: atoms
- Implicit order of precedence (no parentheses needed)



Arithmetic Expressions – Syntax

- The syntax of a programming language is an inductive set
- Defined recursively via a Backus-Naur Form grammar

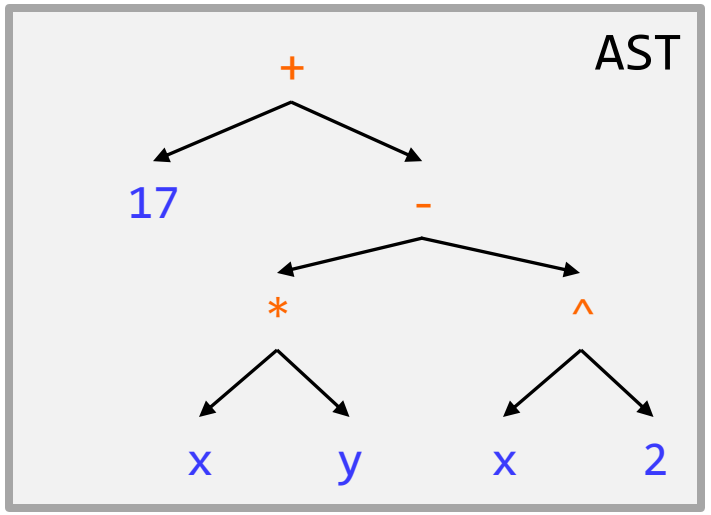
Syntax of Arithmetic Expressions a.

```
a ::= n                          (int constants are atoms)
    | x                          (variables are atoms)
    | a1 + a2 | a1 - a2          (arithmetic operations for
    | a1 * a2 | a1 / a2           composing expressions)
    | a1 ^ a2
```

integer division (rounds down!)

- BNF grammars specify **abstract syntax trees** (ASTs)
  - Nodes: syntactic operators
  - Leaves: atoms
  - Implicit order of precedence

```
17 + (x * y - x ^ 2)
```



Boolean Expressions (Guards) – Syntax

- The syntax of a programming language is an inductive set
- Defined recursively via a Backus-Naur Form grammar

Syntax of Boolean Expressions b.

```
b ::= true                                   (atom)
    | a1 = a2                  (equality between expressions)
    | a1 > a2 | a1 >= a2                     (inequalities)
    | ¬b1                                (logical negation)
    | b1 ∧ b2                        (logical conjunction)
    | b1 && b2                 (short-circuit conjunction)
```

- BNF grammars specify **abstract syntax trees** (ASTs)
  - Nodes: syntactic operators
  - Leaves: atoms
  - Implicit order of precedence

```
true && 42 > x + 17
```

AST

The inequality `42 > x + 17` is an atomic Boolean expression but composed from arithmetic expressions

Exercise

- Construct a Guarded Commands program that computes the power `2^n` of a given natural number n without using the exponentiation expression `a1^a2`.
- Construct the AST of your program

Syntax of Commands `C` and Guarded Commands `GC`.

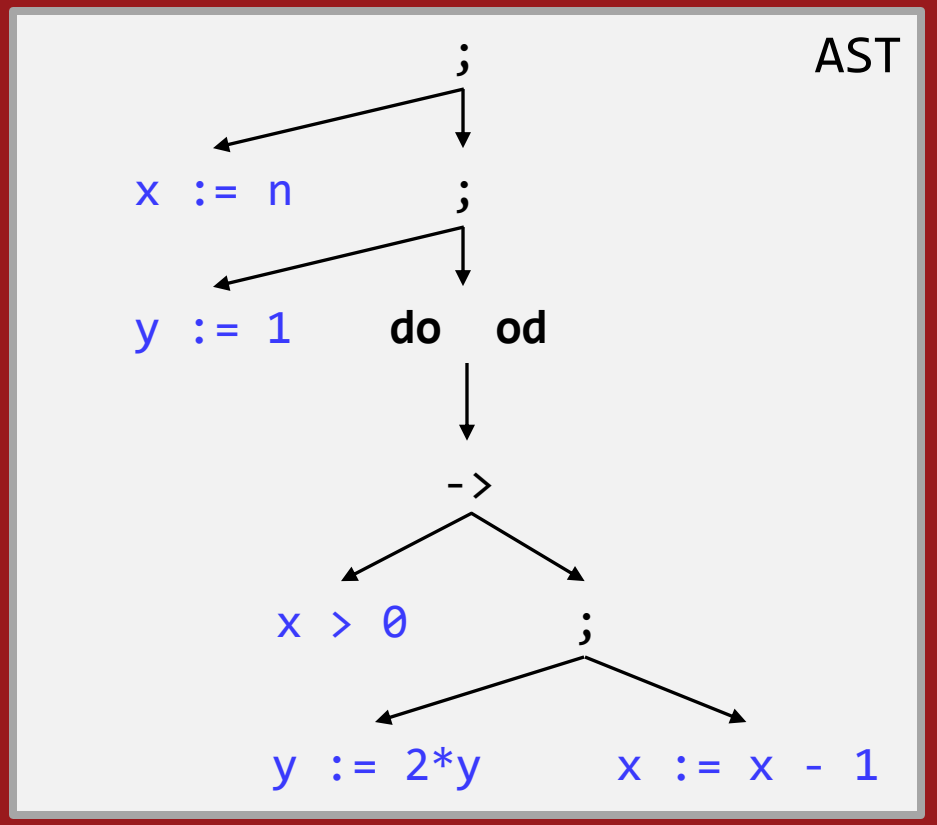
```
C  ::= x := a                 (assignments are atoms)
     | skip                   (no operation is an atom)
     | C1; C2                 (sequential composition)
     | if GC fi               (conditional)
     | do GC od               (iteration)

GC ::= b -> C                 (guarded commands)
     | GC1 [] GC2             (choice)
```

Solution

- Construct a Guarded Commands program that computes the power 2^n of a given natural number n without using the exponentiation expression a1^a2.
- Construct the AST of your program.

```
x := n;
y := 1;
do x > 0 -> y := 2 * y;
            x := x - 1
od
```



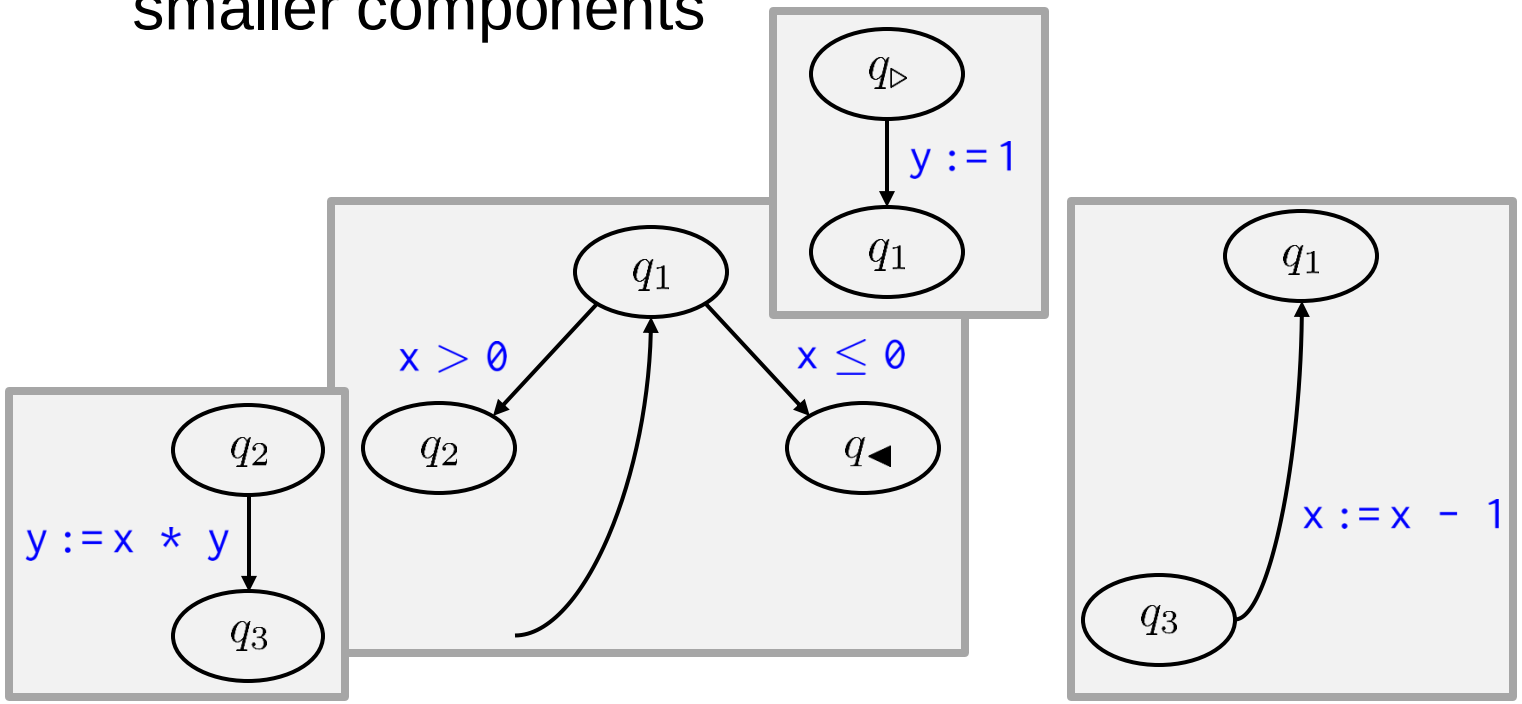
02141 – Computer Science Modelling

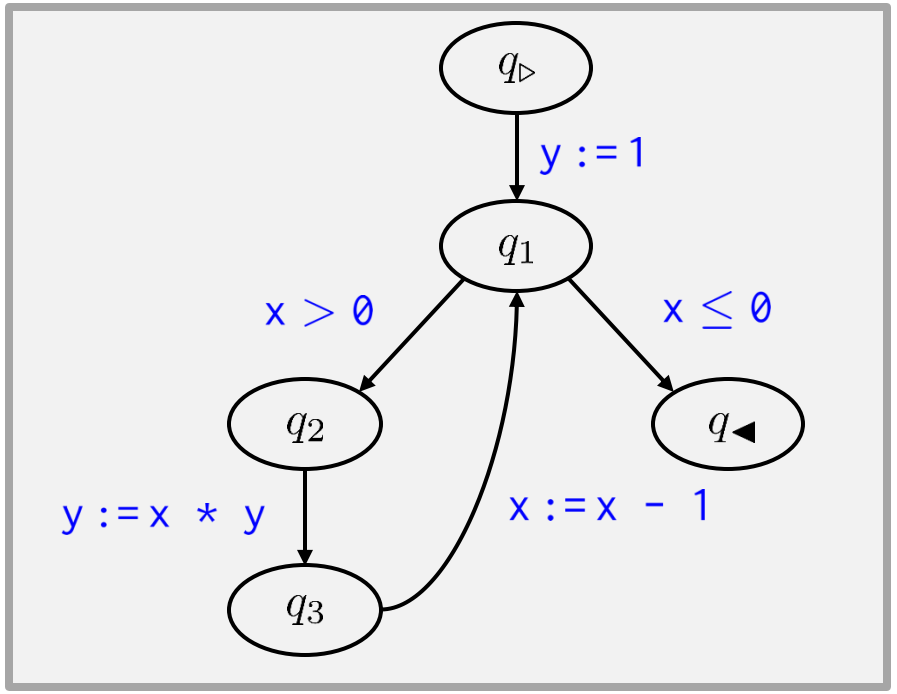
THE GUARDED COMMAND LANGUAGE ➔ PROGRAM GRAPHS

Example

- **Goal:** translate Guarded Command programs into program graphs
- Compositional approach: construct program graphs recursively from smaller components

```
y := 1;
do x > 0 -> y := x * y;
            x := x - 1
od
```





Construction of Program Graphs – Main Idea

- $\mathbf{PG} = (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- set of actions **Act**: `skip`, assignments `x := a` and Boolean expressions `b` in the program
- set of edges **E**: *defined recursively on the program structure*, i.e. its AST
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add nodes on demand
- final result: $\mathbf{E} = \mathbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \mathtt{C} \,]\!]$

source node

command

$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\, \mathtt{C} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node

Construction of Program Graphs – Skip

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- edges **E**: defined recursively on the program structure
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

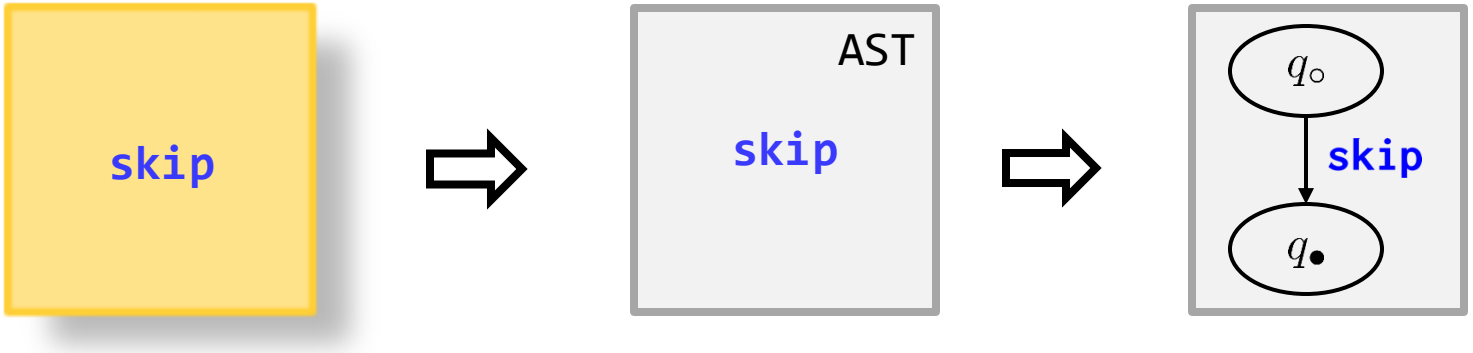
source node

command

$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\, \mathtt{C} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node



$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\, \mathtt{skip} \,]\!] \quad = \quad \{\, (q_{\circ}, \mathtt{skip}, q_{\bullet}) \,\}$$

Construction of Program Graphs – Assignments

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- edges **E**: defined recursively on the program structure
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

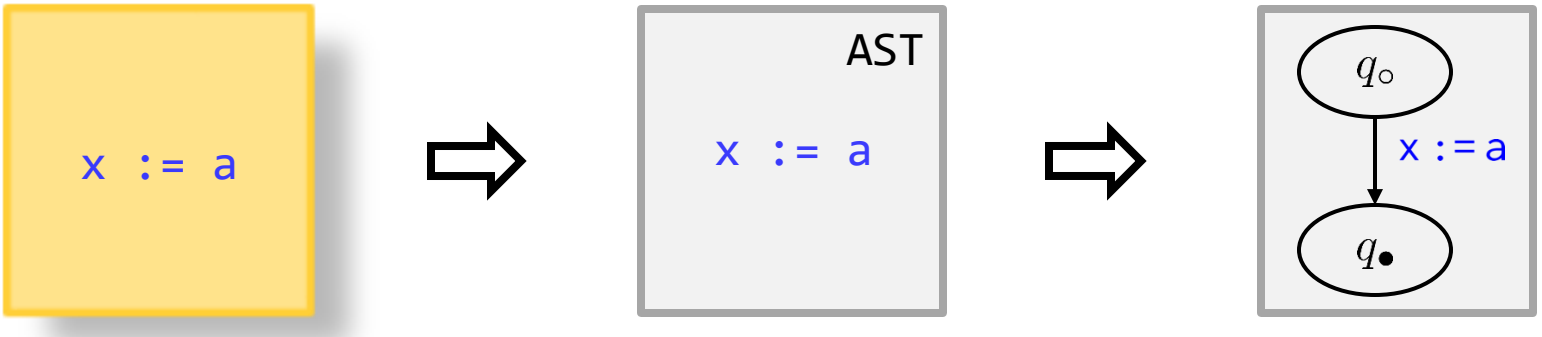
source node

command

$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\, \mathtt{c} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node



$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\, \mathtt{x := a} \,]\!] \quad = \quad \{\, (q_{\circ}, \mathtt{x := a}, q_{\bullet}) \,\}$$

Construction of Program Graphs – Guarded Commands

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- edges **E**: defined recursively on the program structure
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

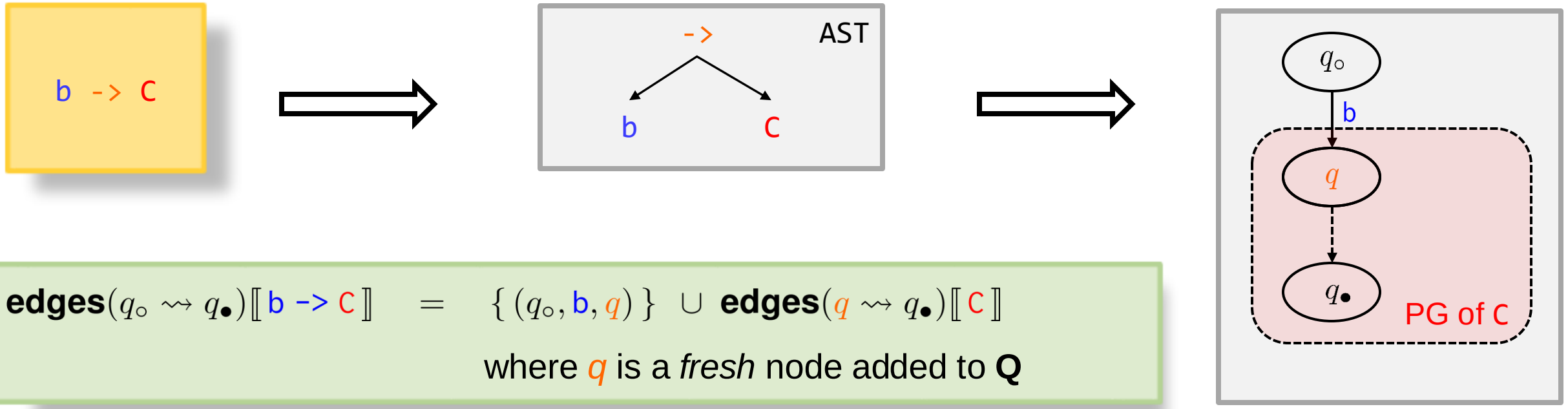
source node

command

$$\mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{C} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node



$$\mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{b \text{ -> } C} \,]\!] \quad = \quad \{\, (q_\circ, \mathtt{b}, q) \,\} \;\cup\; \mathbf{edges}(q \rightsquigarrow q_\bullet)[\![\, \mathtt{C} \,]\!]$$

where q is a *fresh* node added to **Q**

Construction of Program Graphs – Sequential Composition

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- edges **E**: defined recursively on the program structure
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

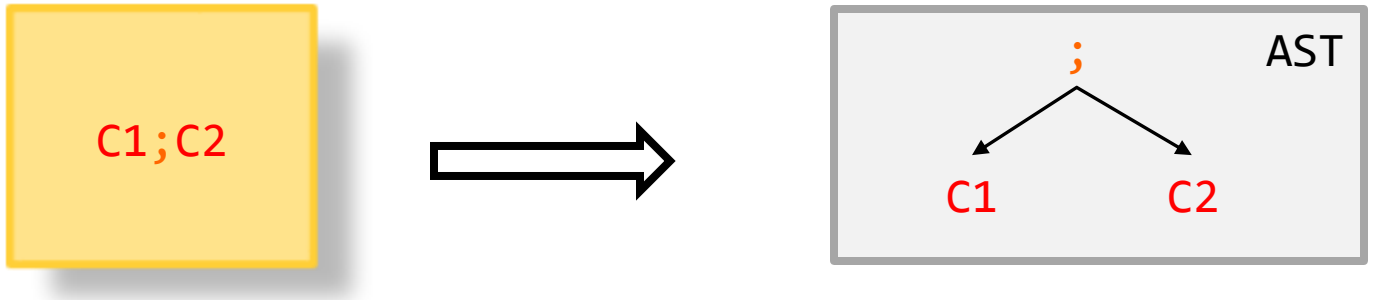
source node

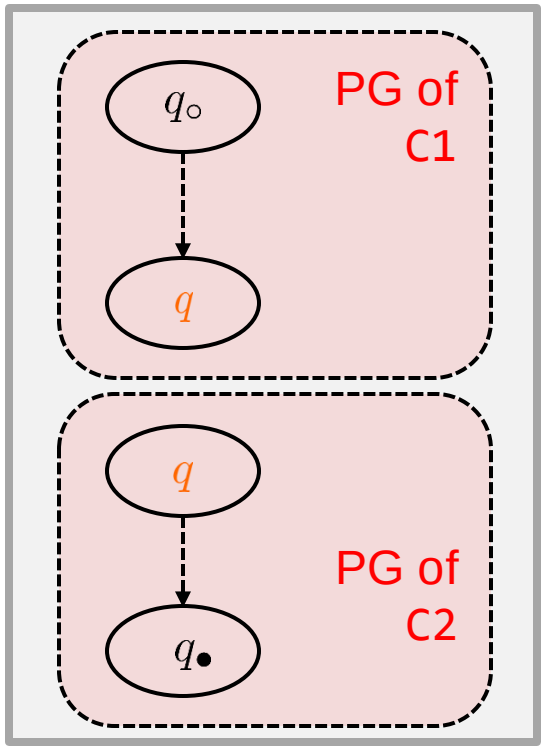
command

$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\mathtt{C}]\!] \subseteq \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node





Construction of Program Graphs – Sequential Composition

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- edges **E**: defined recursively on the program structure
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

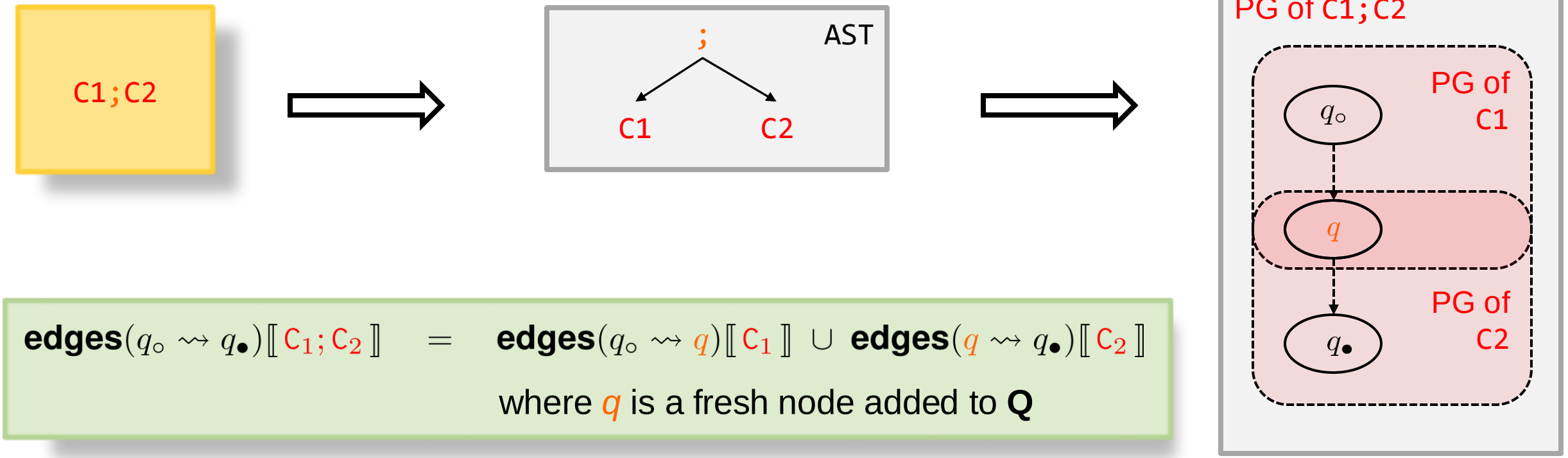
source node

command

$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\, \mathtt{C} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node



$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\, \mathtt{C}_1;\mathtt{C}_2 \,]\!] \quad = \quad \mathbf{edges}(q_{\circ} \rightsquigarrow q)[\![\, \mathtt{C}_1 \,]\!] \cup \mathbf{edges}(q \rightsquigarrow q_{\bullet})[\![\, \mathtt{C}_2 \,]\!]$$

where q is a fresh node added to **Q**

Construction of Program Graphs – Choice

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- edges **E**: defined recursively on the program structure
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

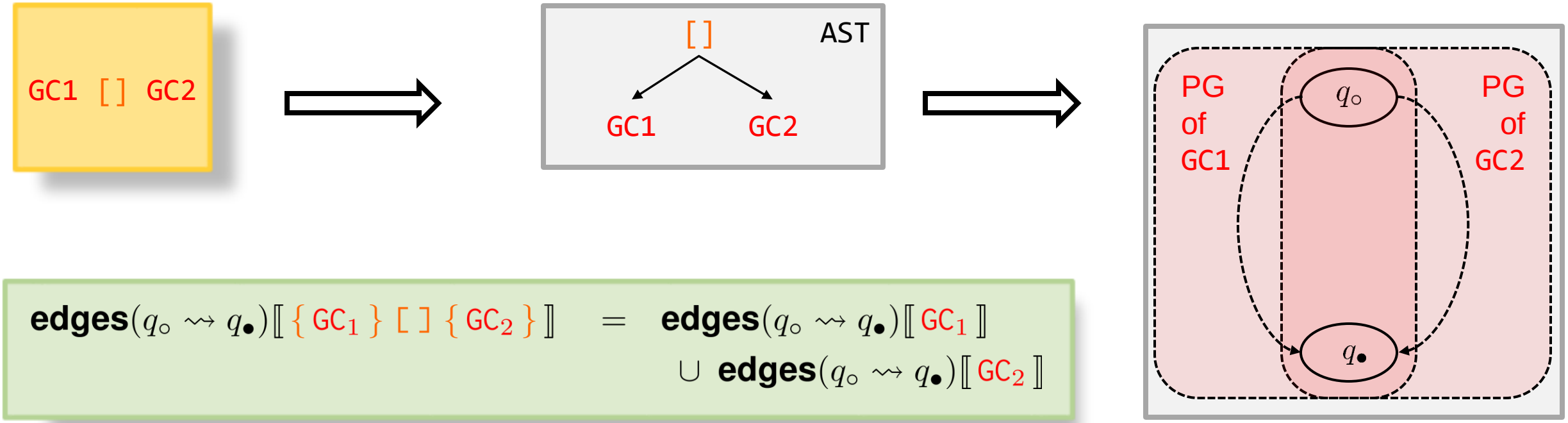
source node

command

$$\mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{C} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node



$$\mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \{\, \mathtt{GC}_1 \,\}\ [\,]\ \{\, \mathtt{GC}_2 \,\} \,]\!] \quad = \quad \mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{GC}_1 \,]\!] \cup \mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{GC}_2 \,]\!]$$

Construction of Program Graphs – Conditional

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- edges **E**: defined recursively on the program structure
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

source node

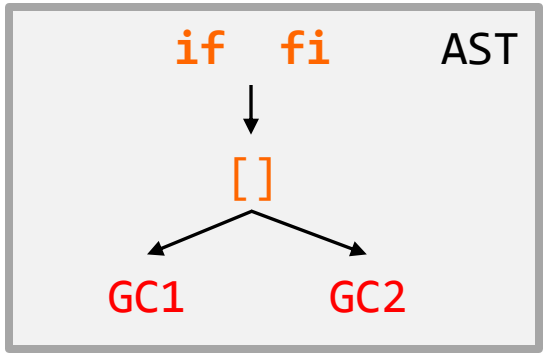
command

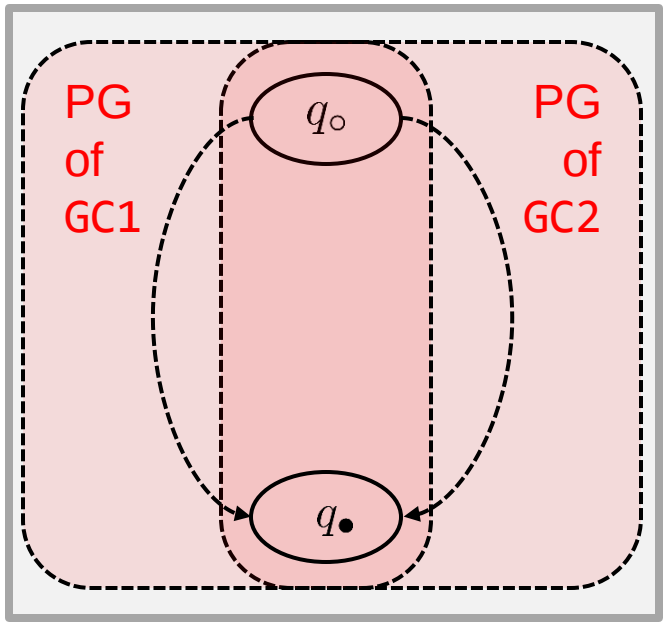
$$\mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{C} \,]\!] \subseteq \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node

```
if
   GC1
[] GC2
fi
```



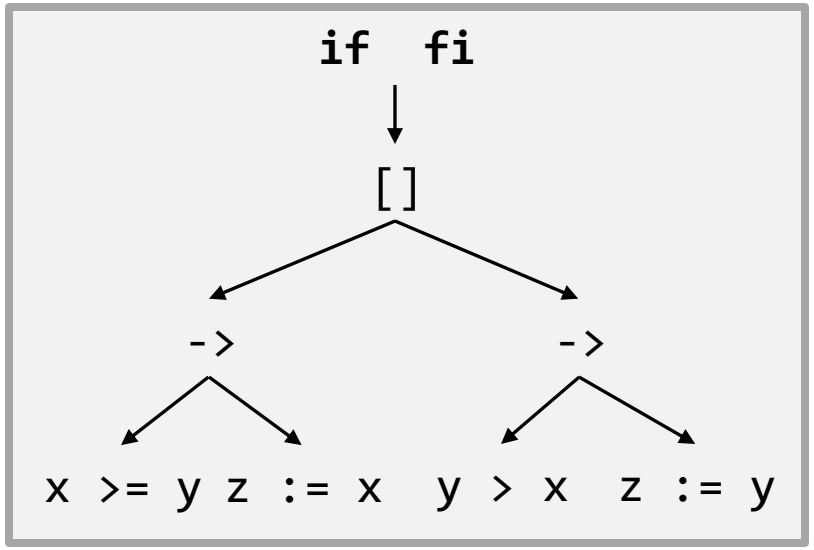


$$\mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{if\ GC\ fi} \,]\!] = \mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{GC} \,]\!]$$

Example

$$\mathbf{E} \;=\; \mathbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \mathtt{C} \,]\!]$$

```
if x >= y -> z := x
[] y > x  -> z := y
fi
```



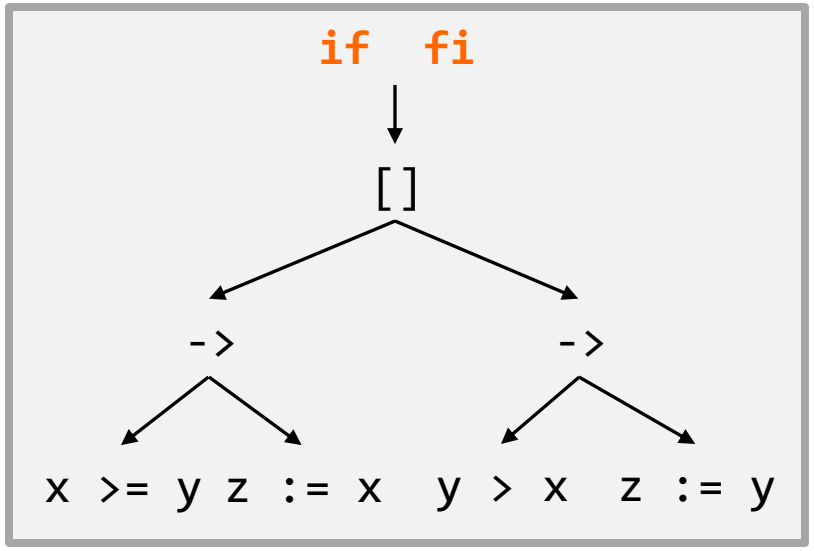
$q_{\triangleright}$

$q_{\blacktriangleleft}$

Example

$$\textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\texttt{if GC fi}]\!] \quad = \quad \textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\texttt{GC}]\!]$$

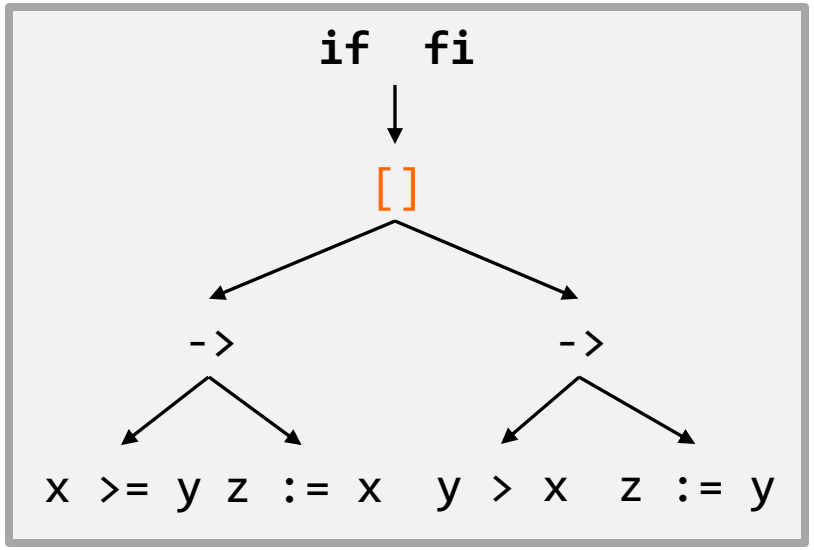
```
if x >= y -> z := x
[] y > x  -> z := y
fi
```



Example

$$\textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \{\, \mathtt{GC}_1 \,\}\ [\,]\ \{\, \mathtt{GC}_2 \,\} \,]\!] \quad = \quad \textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \mathtt{GC}_1 \,]\!] \cup \textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \mathtt{GC}_2 \,]\!]$$

```
if x >= y -> z := x
[] y > x  -> z := y
fi
```

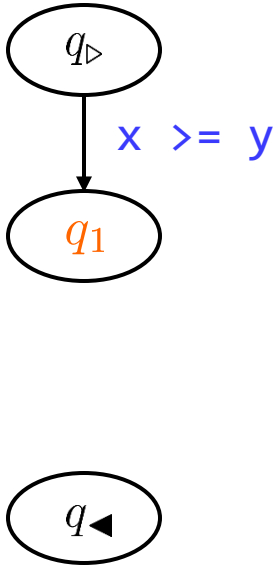


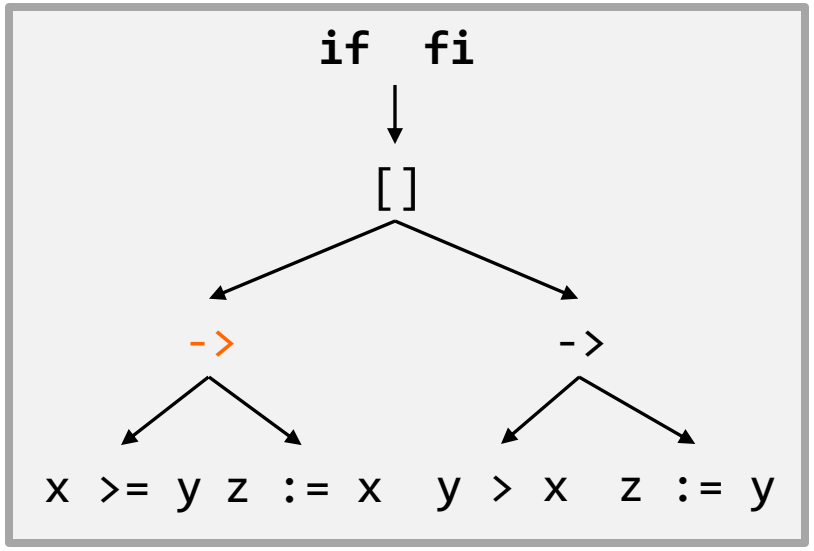
Example

$$\textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\texttt{b -> C}]\!] \quad = \quad \{(q_{\triangleright}, \texttt{b}, q_1)\} \cup \textbf{edges}(q_1 \rightsquigarrow q_{\blacktriangleleft})[\![\texttt{C}]\!]$$

where q_1 is a fresh node added to **Q**

```
if x >= y -> z := x
[] y > x  -> z := y
fi
```

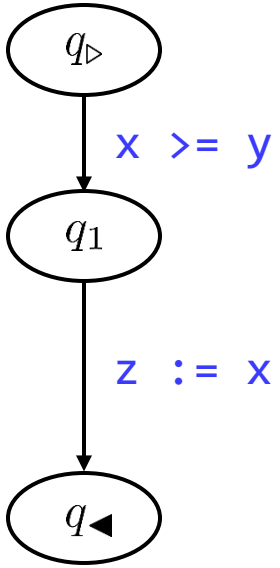


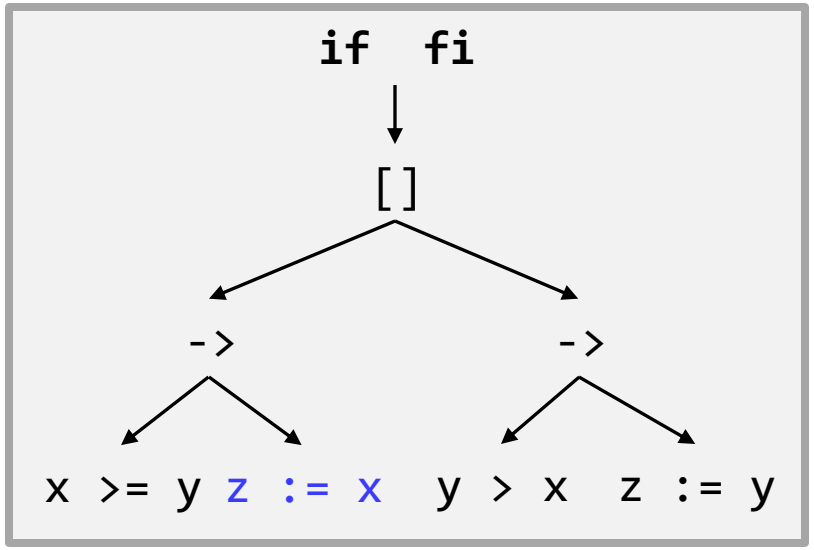


Example

$$\textbf{edges}(q_1 \rightsquigarrow q_{\blacktriangleleft})[\![\texttt{z := x}]\!] \quad = \quad \{\,(q_1, \texttt{z := x}, q_{\blacktriangleleft})\,\}$$

```
if x >= y -> z := x
[] y > x  -> z := y
fi
```

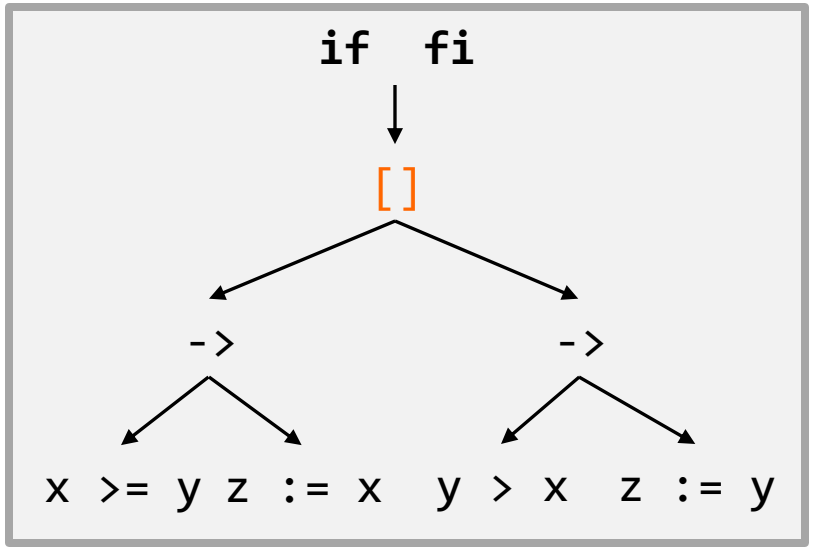


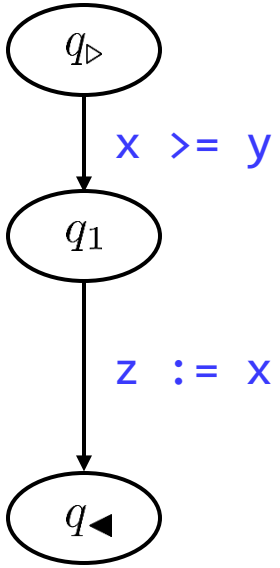


Example

$$\textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \{\, \texttt{GC}_1 \,\}\ [\,]\ \{\, \texttt{GC}_2 \,\} \,]\!] \quad = \quad \textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \texttt{GC}_1 \,]\!] \cup \textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \texttt{GC}_2 \,]\!]$$

```
if x >= y -> z := x
[] y > x  -> z := y
fi
```



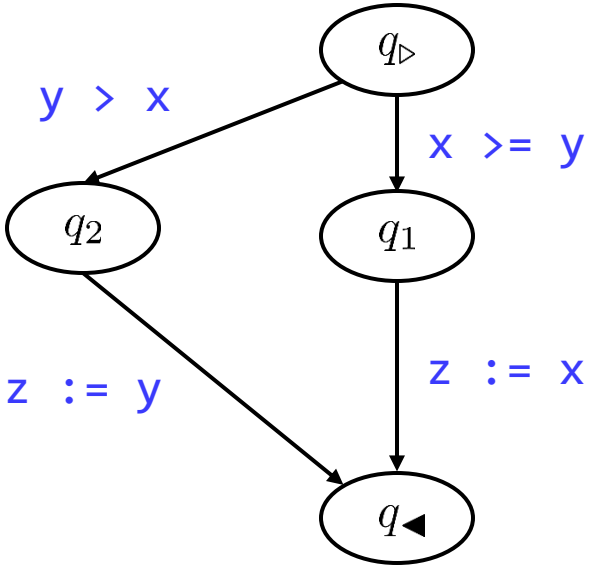


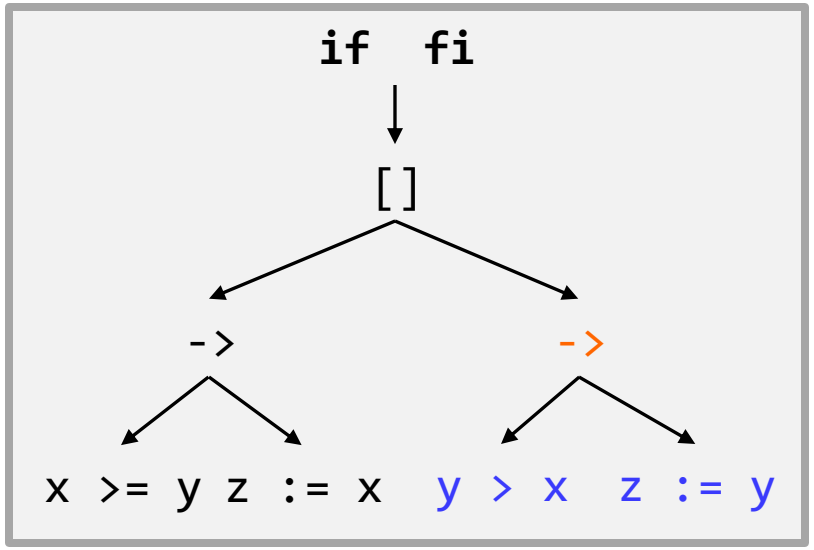
Example

$$\textbf{edges}(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\texttt{b -> C}]\!] \quad = \quad \{\,(q_{\triangleright}, \texttt{b}, q_2)\,\} \;\cup\; \textbf{edges}(q_2 \rightsquigarrow q_{\blacktriangleleft})[\![\texttt{C}]\!]$$

where q_2 is a fresh node added to **Q**

```
if x >= y -> z := x
[] y > x  -> z := y
fi
```





$$\textbf{edges}(q_2 \rightsquigarrow q_{\blacktriangleleft})[\![\texttt{z := y}]\!] \quad = \quad \{\,(q_2, \texttt{z := y}, q_{\blacktriangleleft})\,\}$$

Construction of Program Graphs – Iteration

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- edges **E**: defined recursively on the program structure
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

source node

command

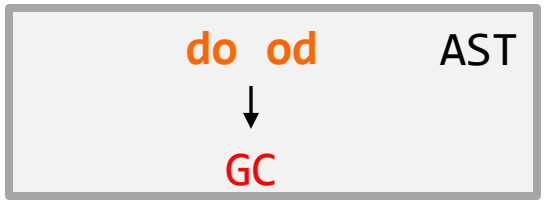
$$\mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{C} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

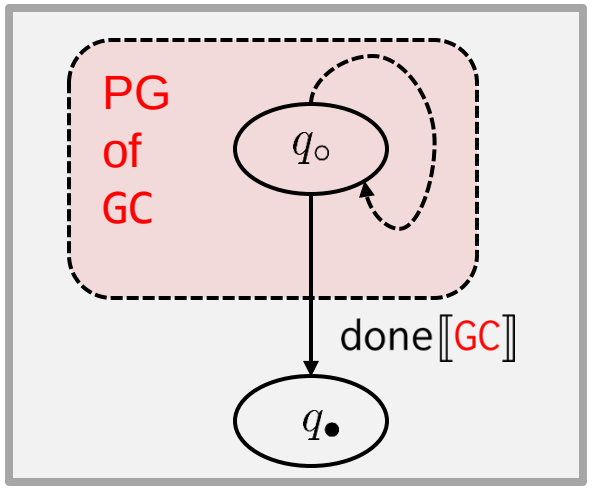
set of program graph edges

target node

---

do GC od





$$\mathbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{do\ GC\ od} \,]\!] \quad = \quad \mathbf{edges}(q_\circ \rightsquigarrow q_\circ)[\![\, \mathtt{GC} \,]\!] \cup \{\, (q_\circ, \mathrm{done}[\![\mathtt{GC}]\!], q_\bullet) \,\}$$

$$\mathrm{done}[\![\mathtt{b\ ->\ C}]\!] \quad = \quad \neg \mathtt{b}$$

$$\mathrm{done}[\![\{\, \mathtt{GC}_1 \,\}\ [\ ]\ \{\, \mathtt{GC}_2 \,\}]\!] \quad = \quad \mathrm{done}[\![\mathtt{GC}_1]\!] \ \wedge\ \mathrm{done}[\![\mathtt{GC}_2]\!]$$

Construction of Program Graphs – Summary

- **PG** $= (\mathbf{Q}, q_{\triangleright}, q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$
- **Act**: assignments and Boolean expressions in the program
- **E** $=$ **edges**$(q_{\triangleright} \rightsquigarrow q_{\blacktriangleleft})[\![\, \mathtt{C} \,]\!]$
- nodes **Q**: start with $q_{\triangleright}, q_{\blacktriangleleft}$; add more nodes on demand

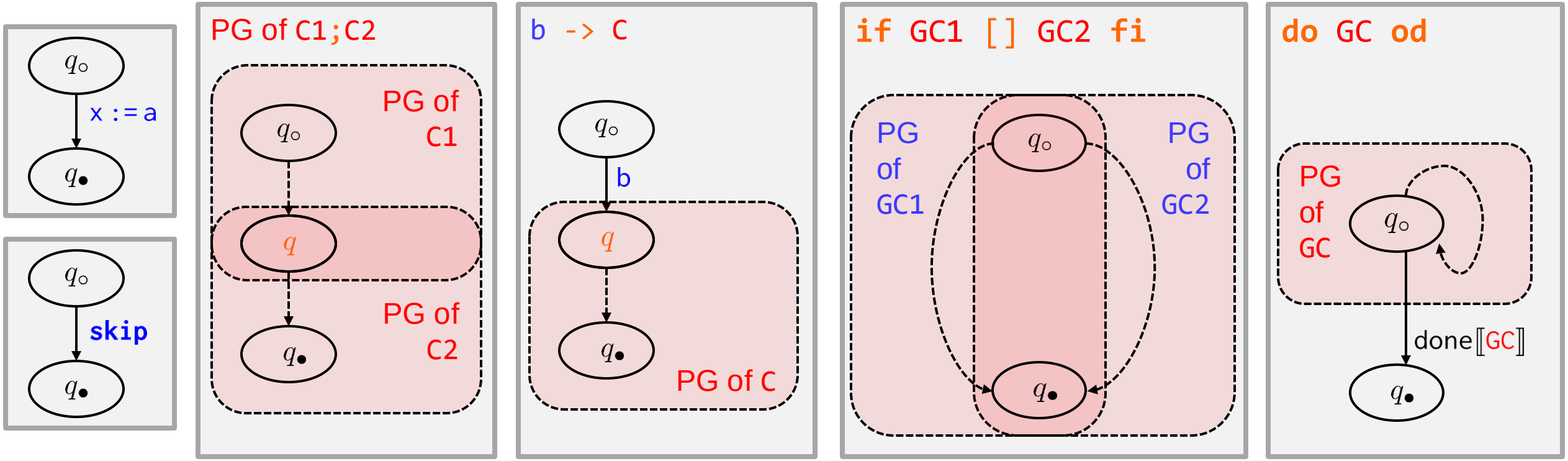
source node

command

$$\mathbf{edges}(q_{\circ} \rightsquigarrow q_{\bullet})[\![\, \mathtt{C} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

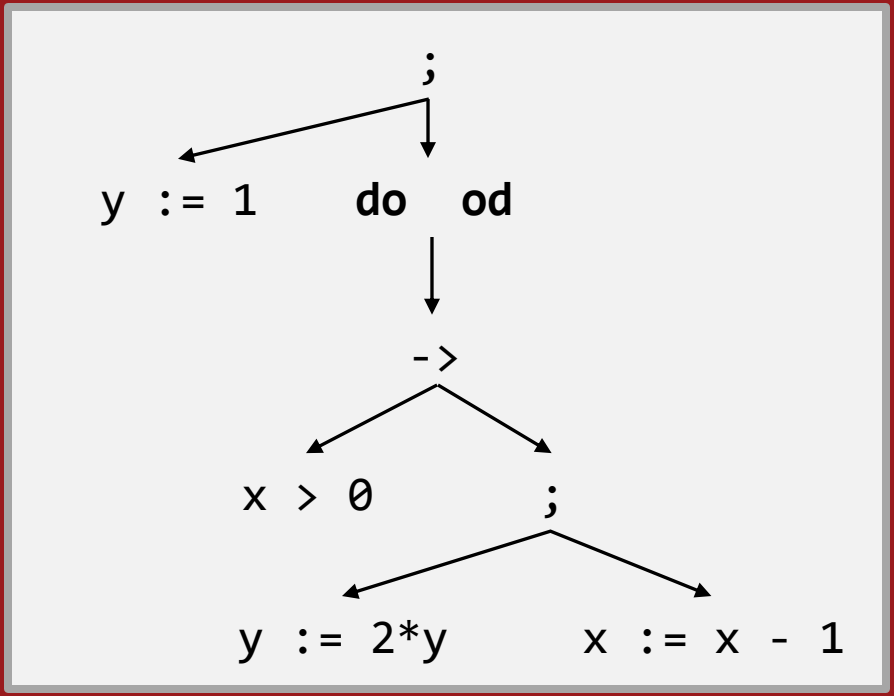
target node



Exercise

- Compute the program graph of the following Guarded Command program

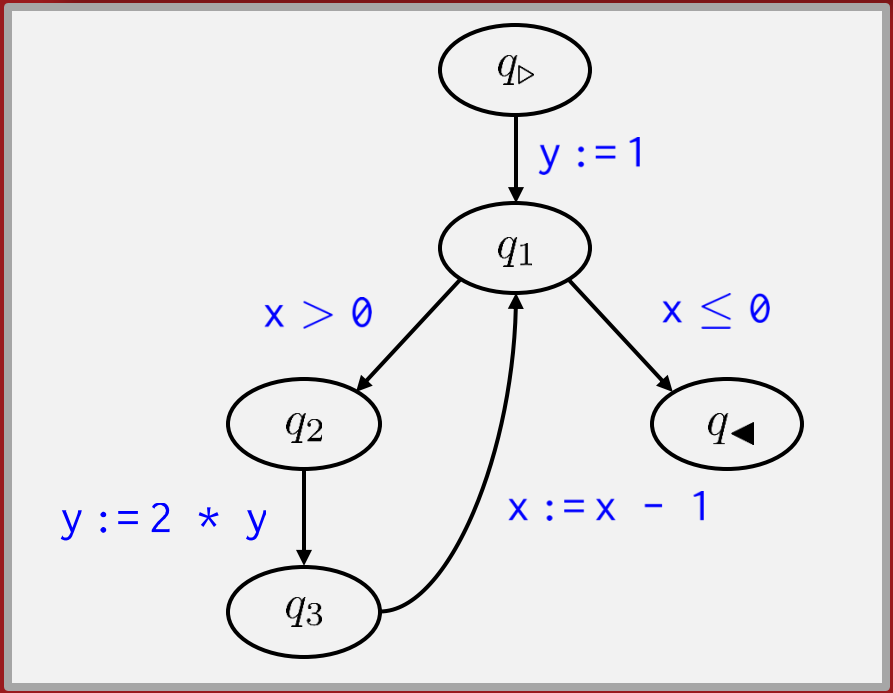
```
y := 1;
do x > 0 -> y := 2 * y;
            x := x - 1
od
```

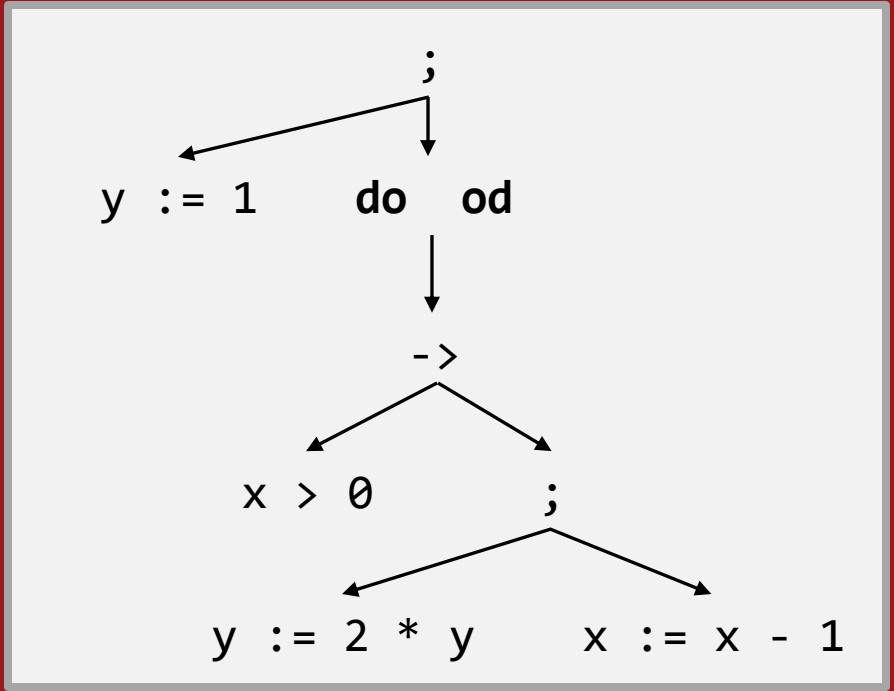


Solution

- Compute the program graph of the following Guarded Command program

```
y := 1;
do x > 0 -> y := 2 * y;
            x := x - 1
od
```





FormalMethods.dk/fm4fun

fm4fun | About | Download | Environments | Formal Methods

Input Program

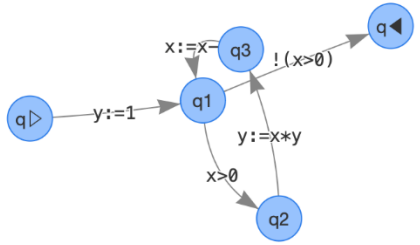
Examples

Extended Guarded Commands Code

```
y:=1;
do x>0 -> y:=x*y;
          x:=x-1
od
```

Program Graph

Non-det. | Det.



Step-wise Execution

Explore how the memory changes, upon execution of the actions. Recall that if more guards are satisfied then it is non-deterministic which action to take.

When using the **Show Trace** feature, non-deterministic choices will be shown in the first column as ⤮, and if clicked the trace will be "reset" from that configuration.

When using the **Show Resulting Configurations** feature, all non-deterministic choices will be explored, until it reaches the number of steps, gets stuck or terminates. A short summary of the final configurations is found at the end. **Warning** If the input code is highly non-deterministic, and the number of steps is high, the tree of possible end-configurations will grow big (and therefore cause long execution).

For both outputs, all non-terminated and non-stuck configurations can be extended with 1-1000 steps.

Initialization of Variables and Arrays

x = 3, y = 0

Number of Steps

10

Show Trace | Show Resulting Configurations

	Configuration				
		Memory			
Steps	Node	x	y	Status	
	q▷	3	0	Initial	
$\Rightarrow^{10}$	q1	0	6	Show More	10 +

1 trace can **Show More** after 10 steps.

02141 – Computer Science Modelling

THE GUARDED COMMAND LANGUAGE ➔ SEMANTICS

Actions of Guarded Commands

Syntax of Commands C and Guarded Commands GC.

```
C  ::= x := a                  (assignments are atoms)
     | skip                    (no operation is an atom)
     | C1; C2                  (sequential composition)
     | if GC fi                (conditional)
     | do GC od                (iteration)

GC ::= b -> C                  (guarded commands)
     | GC1 [] GC2              (choice)
```

Possible actions:

- `x := a`
- `skip`
- `b`

Definition (**semantic function**).

$\mathcal{S}[\![\cdot]\!]\colon \textbf{Act} \to (\textbf{Mem} \hookrightarrow \textbf{Mem})$

Goal: define semantic function of Guarded Command programs *once and for all*

Examples

$$\mathcal{S}[\![\,\mathtt{x} > \mathtt{0}\,]\!](\sigma) \;=\; \begin{cases} \sigma, & \text{if } \sigma(\mathsf{x}) > 0 \\ \text{undefined}, & \text{if } \sigma(\mathsf{x}) \leq 0 \end{cases}$$

$$\mathcal{S}[\![\,\mathtt{x} := \mathtt{x} - \mathtt{1}\,]\!](\sigma) \;=\; \sigma[\mathsf{x} \mapsto \sigma(\mathsf{x}) - 1]$$

Semantics of Actions (informally)

Definition (semantic function).

$\mathcal{S}[\![\cdot]\!]\colon \textbf{Act} \to (\textbf{Mem} \hookrightarrow \textbf{Mem})$

$$\mathcal{S}[\![\texttt{skip}]\!](\sigma) \;=\; \sigma$$

$$\mathcal{S}[\![\texttt{x := a}]\!](\sigma) \;=\; \begin{cases} \sigma[\mathsf{x} \mapsto v], & \text{if } v \text{ is the value obtained from evaluating } \texttt{a} \text{ in memory } \sigma \\ \text{undefined}, & \text{otherwise} \end{cases}$$

$$\mathcal{S}[\![b]\!](\sigma) \;=\; \begin{cases} \sigma, & \text{if } b \text{ evaluates to true in memory } \sigma \\ \text{undefined}, & \text{otherwise} \end{cases}$$

➔ need to formalize evaluation of expressions

Evaluation of Arithmetic Expressions in Memory (incomplete)

Idea: recursively evaluate the AST of an arithmetic expression in a given memory using an evaluation function $\mathcal{A}[\![a]\!] : \mathbf{Mem} \hookrightarrow \mathbf{Int}$

$$\mathcal{A}[\![\mathsf{n}]\!](\sigma) \;=\; n \qquad \text{for every integer constant } n$$

expression *number*

$$\mathcal{A}[\![\mathsf{x}]\!](\sigma) \;=\; \sigma(\mathsf{x})$$

mathematical addition

$$\mathcal{A}[\![a_1 \;+\; a_2]\!](\sigma) \;=\; \mathcal{A}[\![a_1]\!](\sigma) \;+\; \mathcal{A}[\![a_2]\!](\sigma)$$

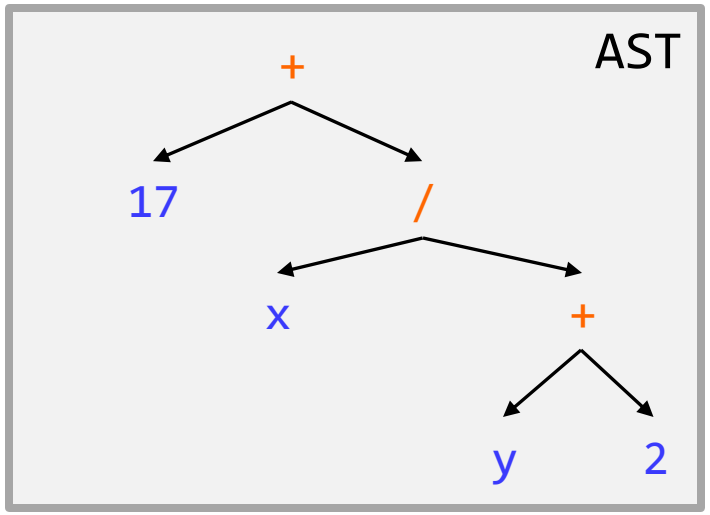
$$\mathcal{A}[\![a_1 \;/\; a_2]\!](\sigma) \;=\; \begin{cases} z, & \text{if } z_1 = \mathcal{A}[\![a_1]\!](\sigma) \text{ and } z_2 = \mathcal{A}[\![a_2]\!](\sigma) \\ & \quad z = z_1 / z_2 \text{ is defined} \\ \text{undefined}, & \text{otherwise} \end{cases}$$

integer division, undefined for div. by 0

Example

```
17 + (x / (y + 2))
```

v ∈ **Var**	σ(v)
x	13
y	4



$$\mathcal{A}[\![17 + (\mathsf{x} \,/\, (\mathsf{y} + 2))]\!](\sigma)$$

$$= \mathcal{A}[\![17]\!](\sigma) \;+\; \mathcal{A}[\![(\mathsf{x} \,/\, (\mathsf{y} + 2))]\!](\sigma)$$

$$= \mathcal{A}[\![17]\!](\sigma) \;+\; (\mathcal{A}[\![\mathsf{x}]\!](\sigma) \,/\, \mathcal{A}[\![\mathsf{y} + 2]\!](\sigma))$$

$$= \mathcal{A}[\![17]\!](\sigma) \;+\; (\mathcal{A}[\![\mathsf{x}]\!](\sigma) \,/\, (\mathcal{A}[\![\mathsf{y}]\!](\sigma) \;+\; \mathcal{A}[\![2]\!](\sigma)))$$

$$= 17 \;+\; (13 \,/\, (4 + 2)) \;=\; 19$$

Evaluation of Arithmetic Expressions in Memory (full)

$$\mathcal{A}[\![\mathsf{n}]\!](\sigma) \;=\; n \qquad \text{for every integer constant } n$$

$$\mathcal{A}[\![\mathsf{x}]\!](\sigma) \;=\; \sigma(\mathsf{x})$$

$$\mathcal{A}[\![a_1 \oplus a_2]\!](\sigma) \;=\; \begin{cases} z, & \text{if } z_1 = \mathcal{A}[\![a_1]\!](\sigma) \text{ and } z_2 = \mathcal{A}[\![a_2]\!](\sigma) \\ & \quad z = z_1 \oplus z_2 \text{ is defined} \\ \text{undefined}, & \text{otherwise} \end{cases}$$

syntactic operator

mathematical operator (possibly undefined: 5 / 0)

where $\oplus \in$ `{ +, -, *, /, ^ }`

Semantics of Assignments

$$\mathcal{S}[\![\mathsf{x := a}]\!](\sigma) \;=\; \begin{cases} \sigma[\mathsf{x} \mapsto v], & \text{if } v \text{ is the value obtained from evaluating } \mathsf{a} \text{ in memory } \sigma \\ \text{undefined}, & \text{otherwise} \end{cases}$$

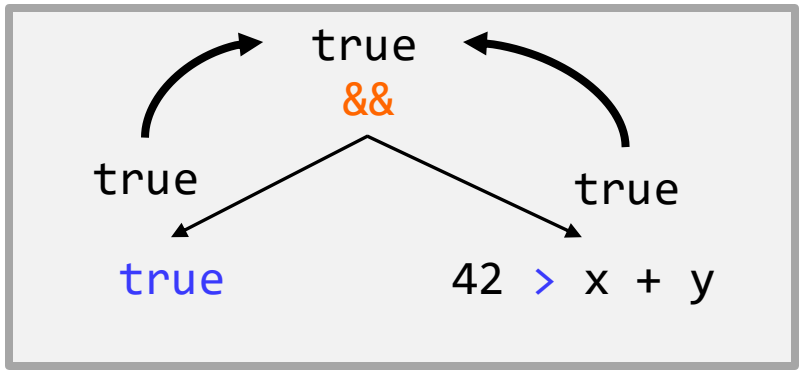
Definition (**semantic function of assignments**).

$$\mathcal{S}[\![\mathsf{x := a}]\!](\sigma) \;=\; \begin{cases} \sigma[\mathsf{x} \mapsto \mathcal{A}[\![\mathsf{a}]\!](\sigma)], & \text{if } \mathcal{A}[\![\mathsf{a}]\!](\sigma) \text{ is defined} \\ \text{undefined}, & \text{otherwise} \end{cases}$$

$$\mathcal{S}[\![\mathsf{x := x - 1}]\!](\sigma) \;=\; \sigma[\mathsf{x} \mapsto \sigma(\mathsf{x}) - 1] \qquad \mathcal{S}[\![\mathsf{y := x * y}]\!](\sigma) \;=\; \sigma[\mathsf{y} \mapsto \sigma(\mathsf{x}) * \sigma(\mathsf{y})]$$

Semantics of Guards

Idea: recursively evaluate the AST of an arithmetic expression in a given memory using an evaluation function $\mathcal{B}[\![\mathrm{b}]\!] : \mathbf{Mem} \hookrightarrow \mathbf{Bool}$



v ∈ **Var**	σ(v)
x	13
y	4

Definition (**semantic function of guards**).

$$\mathcal{S}[\![\mathrm{b}]\!](\sigma) = \begin{cases} \sigma, & \text{if } \mathcal{B}[\![\mathrm{b}]\!](\sigma) = \texttt{true} \text{ (and thus defined)} \\ \text{undefined}, & \text{otherwise} \end{cases}$$

Exercise

- Give an (ideally concise) recursive definition of the evaluation function for Boolean expressions $\mathcal{B}[\![\mathrm{b}]\!] : \mathbf{Mem} \hookrightarrow \mathtt{Bool}$

Syntax of Boolean Expressions b.

```
b ::= true                                        (atom)
    | a1 = a2                  (equality between expressions)
    | a1 > a2 | a1 >= a2                      (inequalities)
    | ¬b1                                (logical negation)
    | b1 ∧ b2                        (logical conjunction)
    | b1 && b2                 (short-circuit conjunction)
```

$$\mathcal{B}[\![\mathtt{true}]\!](\sigma) \;=\; \mathtt{true}$$

Solution

$$\mathcal{B}[\![true]\!](\sigma) = \text{true}$$

$$\mathcal{B}[\![a_1 = a_2]\!](\sigma) = \begin{cases} v, & \text{if } v = (\mathcal{A}[\![a_1]\!](\sigma) = \mathcal{A}[\![a_2]\!](\sigma)) \neq \text{undefined} \\ \text{undefined}, & \text{otherwise} \end{cases}$$

$$\mathcal{B}[\![a_1 < a_2]\!](\sigma) = \begin{cases} true, & \text{if undefined} \neq \mathcal{A}[\![a_1]\!](\sigma) < \mathcal{A}[\![a_2]\!](\sigma) \neq \text{undefined} \\ false, & \text{if undefined} \neq \mathcal{A}[\![a_1]\!](\sigma) \geq \mathcal{A}[\![a_2]\!](\sigma) \neq \text{undefined} \\ \text{undefined}, & \text{otherwise} \end{cases}$$

Solution

$$\mathcal{B}[\![\neg \mathsf{b}]\!](\sigma) = \begin{cases} \texttt{false}, & \text{if } \mathcal{B}[\![\mathsf{b}]\!](\sigma) = \texttt{true} \\ \texttt{true}, & \text{if } \mathcal{B}[\![\mathsf{b}]\!](\sigma) = \texttt{false} \end{cases}$$

$$\mathcal{B}[\![\mathsf{b}_1 \wedge \mathsf{b}_2]\!](\sigma) = \begin{cases} \texttt{true}, & \text{if } \mathcal{B}[\![\mathsf{b}_1]\!](\sigma) = \mathcal{B}[\![\mathsf{b}_2]\!](\sigma) = \texttt{true} \\ \texttt{false}, & \text{if } \mathcal{B}[\![\mathsf{b}_1]\!](\sigma) = \texttt{false} \text{ or } \mathcal{B}[\![\mathsf{b}_2]\!](\sigma) = \texttt{false} \\ \text{undefined}, & \mathcal{B}[\![\mathsf{b}_1]\!](\sigma) = \text{undefined or } \mathcal{B}[\![\mathsf{b}_2]\!](\sigma) = \text{undefined} \end{cases}$$

$$\mathcal{B}[\![\mathsf{b}_1 \,\&\&\, \mathsf{b}_2]\!](\sigma) = \begin{cases} \mathcal{B}[\![\mathsf{b}_2]\!](\sigma), & \text{if } \mathcal{B}[\![\mathsf{b}_1]\!](\sigma) = \texttt{true} \\ \texttt{false}, & \text{if } \mathcal{B}[\![\mathsf{b}_1]\!](\sigma) = \texttt{false} \\ \text{undefined}, & \mathcal{B}[\![\mathsf{b}_1]\!](\sigma) = \text{undefined} \end{cases}$$

Wrap-Up

The semantics of actions and expressions is defined recursively on their structure (AST)

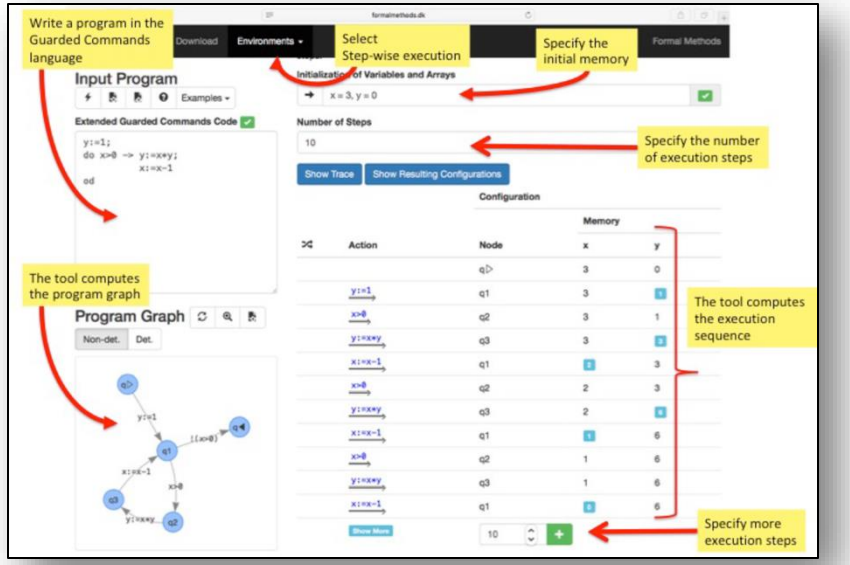
Definition (**semantic function of skip**).

$$\mathcal{S}[\![\texttt{skip}]\!](\sigma) \;=\; \sigma$$

Definition (**semantic function of assignments**).

$$\mathcal{S}[\![\mathsf{x} := \mathsf{a}]\!](\sigma) \;=\; \begin{cases} \sigma[\mathsf{x} \mapsto \mathcal{A}[\![\mathsf{a}]\!](\sigma)], & \text{if } \mathcal{A}[\![\mathsf{a}]\!](\sigma) \text{ is defined} \\ \text{undefined}, & \text{otherwise} \end{cases}$$

Definition (**semantic function of guards**).

$$\mathcal{S}[\![\mathsf{b}]\!](\sigma) \;=\; \begin{cases} \sigma, & \text{if } \mathcal{B}[\![\mathsf{b}]\!](\sigma) = \texttt{true} \text{ (and thus defined)} \\ \text{undefined}, & \text{otherwise} \end{cases}$$

Definition. The **semantics of a Guarded Command program** is the set of all complete execution sequences of its program graph and its semantic function.